ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 24

Router, peticiones y respuestas

- Del if/elseif a una lista de rutas
- La Petición: todo lo que llega
- La Respuesta: todo lo que sale
- El Router por dentro
- Rutas con parámetros
- 404 y 405: cuando la ruta no encaja
- 500: el manejador global de errores
- El front controller, en cinco líneas



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

En la clase 23 os dejé una lista de cosas que fallaban en nuestro MVC. Hoy tachamos las tres primeras: la cadena de `if/elseif` del front controller se convierte en un **Router**, los `$_GET`, `$_SERVER` y `echo` sueltos pasan a vivir en dos clases, **Peticion** y **Respuesta**, y los errores dejan de enseñarse al visitante gracias al **manejador global** de la clase 22.

Al terminar, el front controller cabrá en cinco líneas. Y la forma de declarar las rutas será prácticamente la de Laravel.

•
👌 Cómo vamos a trabajar



1
🤔 Del if/elseif a una lista de rutas



2
📘 La Petición: todo lo que llega



3
👍 La Respuesta: todo lo que sale



4
🔍 El Router por dentro



5
🚀 Rutas con parámetros



6
⚠️ 404 y 405: cuando la ruta no encaja



7
🐞 500: el manejador global de errores



8
👌 El front controller, en cinco líneas



9
🚀 Ejercicios



10
👋 Lo que tenéis que llevaros de hoy





CLASE 24

👌 Cómo vamos a trabajar

Seguimos con `academia-mvc`. Sustituid su contenido por el de `proyecto/` (o comparad con lo que teníais y copiad lo nuevo) y ejecutad `composer dump-autoload`, porque hay clases nuevas en `src/Nucleo/`.

Esquema

```
src/Nucleo/              ← NUEVO: las piezas del framework
├── Peticion.php
├── Respuesta.php
├── Router.php
└── ManejadorErrores.php
config/rutas.php         ← NUEVO: todas las rutas, de un vistazo
vistas/errores/405.php   ← NUEVO
vistas/errores/500.php   ← NUEVO
```



CLASE 24 · PUNTO 1

🤔 Del if/elseif a una lista de rutas

Así decidía el front controller de la clase 23 a quién llamar:

ejemplo.php — php-paso-a-paso/clase24 — Visual Studio Code

ejemplo.php

```php
<?php
if ($metodo === "GET" && ($ruta === "/" || $ruta === "/cursos")) {
    $html = $controlador->index();
} elseif ($metodo === "GET" && preg_match("#^/cursos/(\d+)$#", $ruta, $partes)) {
    $html = $controlador->ver((int) $partes[1]);
} else {
    // 404...
}
```

Con veinte páginas, veinte `elseif`, cada uno con su expresión regular. Ahora, así:

ejemplo.php — php-paso-a-paso/clase24 — Visual Studio Code

ejemplo.php

```php
<?php
$router->get("/", [CursosControlador::class, "index"]);
$router->get("/cursos", [CursosControlador::class, "index"]);
$router->get("/cursos/{id}", [CursosControlador::class, "ver"]);
```

Cada línea dice: «para una petición **GET** a esta **dirección**, llama a este **método** de este **controlador**». Y todas las rutas de la web están en un solo archivo, `config/rutas.php`, que se lee como un índice [1].



💡 `CursosControlador::class` es el nombre completo de la clase como texto (`"App\Controladores\CursosControlador"`). Lo visteis en la clase 20.



CLASE 24 · PUNTO 2

📘 La Petición: todo lo que llega

En vez de leer `$_SERVER`, `$_GET` y `$_POST` por todas partes, los metemos en un objeto que se crea **una vez** al principio:

Peticion.php — php-paso-a-paso/clase24 — Visual Studio Code

Peticion.php

```php
<?php
namespace App\Nucleo;

class Peticion
{
    public function __construct(
        public readonly string $metodo,
        public readonly string $ruta,
        private array $consulta = [],      // $_GET
        private array $formulario = [],    // $_POST
    ) {
    }

    public static function desdeGlobales(): Peticion
    {
        $ruta = parse_url($_SERVER["REQUEST_URI"] ?? "/", PHP_URL_PATH) ?: "/";
        $ruta = rtrim($ruta, "/") ?: "/";
        return new Peticion(strtoupper($_SERVER["REQUEST_METHOD"] ?? "GET"), $ruta, $_GET,
$_POST);
    }

    public function consulta(string $clave, string $porDefecto = ""): string { /* ... */ }
    public function formulario(string $clave, string $porDefecto = ""): string { /* ... */ }
}
```

En el controlador, en lugar de `trim($_GET["q"] ?? "")`, escribimos `$peticion->consulta("q")`: más corto, ya limpio y sin tocar las superglobales.

📄 Archivo: `src/Nucleo/Peticion.php`



🎯 ¿Por qué tanto lío para leer `$_GET`? Porque una `Peticion` se puede **fabricar** en una prueba (`new Peticion("GET", "/cursos", ["q" => "php"])`) sin navegador ni servidor. Recordad la clase 22: código que se puede probar.



CLASE 24 · PUNTO 3

👍 La Respuesta: todo lo que sale

Lo mismo hacia fuera: el contenido, el código de estado y las cabeceras viajan juntos en un objeto `Respuesta`, con métodos estáticos que **fabrican** los tipos habituales:

Método	Qué devuelve
`Respuesta::html($html, 200)`	Una página HTML
`Respuesta::texto("Hola", 200)`	Texto plano (`Content-Type: text/plain`)
`Respuesta::redirigir("/cursos")`	Un código 302 con la cabecera `Location`

Y un único método, `enviar()`, que es **el único sitio de toda la aplicación** donde se llama a `header()` y a `echo`:

Respuesta.php — php-paso-a-paso/clase24 — Visual Studio Code

Respuesta.php

```php
<?php
public function enviar(): void
{
    http_response_code($this->codigo);
    foreach ($this->cabeceras as $nombre => $valor) {
        header("{$nombre}: {$valor}");
    }
    echo $this->contenido;
}
```

¿Os acordáis del error *headers already sent*? Con este diseño es imposible: nada se envía hasta el final.

Ahora, todas las acciones de los controladores **reciben** una `Peticion` y **devuelven** una `Respuesta`:

Respuesta.php — php-paso-a-paso/clase24 — Visual Studio Code

Respuesta.php

```php
<?php
public function index(Peticion $peticion): Respuesta
{
    $busqueda = $peticion->consulta("q");
    $cursos = $this->cursos->buscar($busqueda, 1, 100);

    return Respuesta::html(vista("cursos/index", [
        "titulo" => "Cursos",
        "cursos" => $cursos,
        "busqueda" => $busqueda,
    ]));
}
```

📄 Archivos: `src/Nucleo/Respuesta.php` y `src/Controladores/CursosControlador.php`



💡 Petición entra, respuesta sale: así funciona **cualquier** *framework* web, en PHP y en otros lenguajes. Laravel tiene sus clases `Request` y `Response`, con la misma idea.



CLASE 24 · PUNTO 4

🔍 El Router por dentro

El router guarda una lista de rutas y, al llegar una petición, busca la primera que encaja:

Router.php — php-paso-a-paso/clase24 — Visual Studio Code

Router.php

```php
<?php
class Router
{
    private array $rutas = [];

    public function get(string $ruta, array|callable $accion): void
    {
        $this->anadir("GET", $ruta, $accion);
    }

    public function post(string $ruta, array|callable $accion): void
    {
        $this->anadir("POST", $ruta, $accion);
    }

    public function despachar(Peticion $peticion): Respuesta
    {
        foreach ($this->rutas as $ruta) {
            if (preg_match($ruta["patron"], $peticion->ruta, $coincidencias) === 1) {
                continue;                                   // no es esta ruta
            }
            // ...si el método también coincide, ejecutamos la acción
            return $this->ejecutar($ruta["accion"], $peticion, $this
->parametros($coincidencias));
        }
        // ...si ninguna encaja: 404 (o 405, punto 6)
    }
}
```

Y para ejecutar la acción, si es un array `[Clase::class, "metodo"]`, crea el controlador y llama al método:

Router.php — php-paso-a-paso/clase24 — Visual Studio Code

Router.php

```php
<?php
[$clase, $metodo] = $accion;
$controlador = new $clase();
return $controlador->$metodo($peticion, ...$parametros);
```

Fijaos en `new $clase()` y `->$metodo(...)`: el nombre de la clase y el del método están **en variables**. PHP permite usarlas así.

📄 Archivo: `src/Nucleo/Router.php`



En la ruta, `{id}` marca un **trozo variable**. El router lo convierte en una expresión regular con un **grupo con nombre** [2]:

Esquema

```
"/cursos/{id}"   →   "#^/cursos/(?P<id>[^/]+)$#"
```

`(?P<id>…)` significa «guarda lo que encaje aquí con el nombre `id`», y `[^/]+` es «uno o más caracteres que no sean `/`». Para `/cursos/5`, `preg_match()` devuelve `["id" ⇒ "5"]`.

Después, el router pasa los parámetros **por su nombre** al método del controlador (convirtiendo a `int` los que son números):

ejemplo.php — php-paso-a-paso/clase24 — Visual Studio Code

ejemplo.php

```php
<?php
$router->get("/cursos/{id}", [CursosControlador::class, "ver"]);

public function ver(Peticion $peticion, int $id): Respuesta { /* ... */ }
```

¿Y si una ruta es tan sencilla que no merece un controlador? Se puede atender con una **función anónima**:

ejemplo.php — php-paso-a-paso/clase24 — Visual Studio Code

ejemplo.php

```php
<?php
$router->get("/hola/{nombre}", function (Peticion $peticion, string $nombre): Respuesta {
    return Respuesta::texto("¡Hola, {$nombre}! 👋");
});
```

Probad `http://academia-mvc.test/hola/Marta`.



CLASE 24 · PUNTO 6

⚠️ 404 y 405: cuando la ruta no encaja

Situación	Código	Ejemplo
Ninguna ruta encaja con la dirección	**404** *Not Found*	`GET /nada`
La dirección existe, pero no con ese método	**405** *Method Not Allowed*	`POST /cursos` (solo hay `GET /cursos`)

Son dos de los códigos de estado de HTTP que vimos en la clase 1 [3]. El router distingue los dos casos: si alguna ruta encajaba por la dirección pero no por el método, responde 405; si ninguna encajaba, 404. Cada uno con su vista en `vistas/errores/`.



💡 Los códigos de estado no son decoración: los navegadores, los buscadores como Google y las aplicaciones que consumen vuestra web los leen. Un 404 bien puesto le dice a Google «esto no existe, no lo indexes».



CLASE 24 · PUNTO 7

🐞 500: el manejador global de errores

Probad `http://academia-mvc.test/cursos/abc`. La ruta `/cursos/{id}` encaja, el router pasa el texto `"abc"`… y `ver(Peticion $peticion, int $id)` espera un entero. Resultado: un `TypeError` que nadie captura.

Para eso registramos el **manejador global** de la clase 22, convertido en una pieza del *framework*:

ManejadorErrores.php — php-paso-a-paso/clase24 — Visual Studio Code

ManejadorErrores.php

```php
<?php
class ManejadorErrores
{
    public static function registrar(bool $depuracion): void
    {
        self::$depuracion = $depuracion;
        ini_set("display_errors", "0");
        set_error_handler([self::class, "avisoAExcepcion"]);
        set_exception_handler([self::class, "manejar"]);
    }

    public static function manejar(Throwable $e): void
    {
        error_log(/* todos los detalles */);
        $html = vista("errores/500", [
            "titulo" => "Error del servidor",
            "detalle" => self::$depuracion ? $e : null,
        ]);
        Respuesta::html($html, 500)->enviar();
    }
}
```

El visitante ve una página 500 amable. Y vosotros, mientras desarrolláis (`DEPURACION = true` en `config/config.php`), veis además el error completo en esa misma página.

📄 Archivos: `src/Nucleo/ManejadorErrores.php` y `vistas/errores/500.php`



⚠️ **`DEPURACION` siempre en `false` en un servidor de verdad.** Los detalles de un error (rutas, nombres de clases, consultas) son una pista estupenda para un atacante. Laravel tiene lo mismo: la opción `APP_DEBUG`.



CLASE 24 · PUNTO 8

👌 El front controller, en cinco líneas

index.php — php-paso-a-paso/clase24 — Visual Studio Code

index.php

```php
<?php
declare(strict_types=1);

require_once __DIR__ . "/../vendor/autoload.php";
require_once __DIR__ . "/../config/config.php";

use App\Nucleo\ManejadorErrores;
use App\Nucleo\Peticion;
use App\Nucleo\Router;

ManejadorErrores::registrar(DEPURACION);

$router = new Router();
require __DIR__ . "/../config/rutas.php";

$peticion = Peticion::desdeGlobales();

$respuesta = $router->despachar($peticion);

$respuesta->enviar();
```

Leedlo de arriba abajo: preparar los errores, cargar las rutas, leer la **petición** (entrada), buscar quién la atiende (procesamiento) y enviar la **respuesta** (salida). Nuestro orden de siempre, a lo grande.

📄 Archivo: `public/index.php`

✅ Pruebas para comprobarlo

Haced esto	Debéis obtener
`/`, `/cursos` y `/cursos?q=php`	El catálogo
`/cursos/2`	La ficha de JavaScript
`/cursos/99`	404: el curso no existe
`/nada`	404: la ruta no existe
`/hola/Marta`	«¡Hola, Marta! 👋» en texto plano
`/cursos/abc`	500, con el detalle del `TypeError` porque `DEPURACION` es `true`
Un formulario que haga `POST` a `/cursos`	405



CLASE 24 · PUNTO 9

🚀 Ejercicios

- ☐ **01 · Precio en texto.** Añadid `GET /cursos/{id}/precio`, que responda en **texto plano** «El curso X cuesta Y con IVA» (o un 404 en texto si no existe). → `CursosControlador.php`
- ☐ **02 · Solo números.** Haced que el router admita restricciones como `{id:\d+}`, para que `/cursos/abc` dé un **404** en lugar de un 500. Pista: `preg_replace_callback()`. → `Router.php` y `config/rutas.php`
- ☐ **03 · Mapa de rutas.** Añadid al router un método `listar()` y una ruta `GET /rutas` que muestre en una tabla todas las rutas registradas y quién las atiende. Pista: una función anónima con `use ($router)`. → `Router.php` y `vistas/rutas.php`



🏆 **Reto:** añadid al router los métodos `put()` y `delete()`. Todavía no podréis usarlos desde un formulario HTML (solo admite GET y POST), pero los necesitaremos para la API REST de la clase 27.



CLASE 24 · PUNTO 10

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Router	Una lista de rutas: método + dirección + quién la atiende
`config/rutas.php`	Todas las rutas de la web, de un vistazo
`Peticion`	Todo lo que llega, en un objeto: método, ruta, `$_GET` y `$_POST`
`Respuesta`	Todo lo que sale: contenido, código y cabeceras; se envía al final
Acciones	Reciben una `Peticion` y devuelven una `Respuesta`
`{id}`	Parámetro de ruta, convertido en grupo con nombre `(?P<id>…)`
404 / 405 / 500	No existe / método no permitido / error del servidor
`DEPURACION`	Detalles del error solo mientras desarrolláis

📚 Fuentes y documentación oficial

1. Laravel: Routing
2. Manual de PHP: sintaxis de los patrones (subpatrones con nombre)
3. MDN Web Docs: códigos de estado de respuesta HTTP